WowCare 2.0

Engineering Standards: Eight Mandates

Navigation, state, structure, theming, layer discipline, reuse, responsiveness, models

REFERENCE	VALUE
Branch	`develop`
Commit	`16b1b2b61ea65cad2ba4e26c9f55e6991f2d6833`
Commit subject	Merge pull request #309 from ChildcareWow/feature/LLDB-Fix
Commit date	24 August 2026
Scope	lib/ (1,389 Dart files, 375,430 lines) and test/ (31 files, 8,462 lines)
Toolchain checks	flutter analyze, flutter test, flutter build bundle, import-graph reachability
Supersedes	wowcare_architecture_standard.docx (May 2026). Target folder structure in Section 3 is reproduced from it unchanged.

Every number and every file:line reference in this document was measured against the commit above. **Nothing here is carried over from the May audit without re-verification.**

The May 2026 standard was not ignored. The domain layer it asked for exists (19 features, 224 use cases, 91 of 115 cubits wired through them), main.dart went from 711 lines to 79, the plaintext password key was removed with a migration path, and ChangeNotifier state management is gone. Resource disposal is disciplined: exactly three files in the codebase create a controller without disposing it, and there is not one uncancelled StreamSubscription. That work was real.

What did not happen was enforcement. The eight mandates below are the parts that need to hold in every file, not just in new features, and Section 9 covers the only mechanism that will make them hold.

0. How to read this document

Each of the eight sections has the same shape:

- **Where we are today**, measured. Counts are exact greps against the reference commit.
- **The rule**, stated so a reviewer can apply it without interpretation.
- **Worked examples** from this codebase. Real files, real line numbers, wrong version and right version side by side.
- **Migration**, what to change and roughly how big it is.
- **Impact on future maintenance**, the specific mechanism at work and the concrete situation a developer runs into because of it, whether they are adding a feature or fixing a bug.

Two conventions used throughout:

- **WRONG** blocks are copied from the codebase as it stands. They are not hypothetical.
- **RIGHT** blocks are the replacement, written against APIs that already exist in this repo.

0.1 Scorecard at the reference commit

Mandate	Violations	Files hit	Single worst offender
1. GoRouter everywhere	**249**	91	`attendance/widgets/attendance_popup.dart (26)`
2. setState scarcely	**1,631**	311	`activities/view/activity_daily_tab.dart (68)`
3. Folder structure	**4 competing layouts**	19 features	`communication/presentation/ nested inside presentation/`
4. Centralized theming	**7,928**	~400	`3,479 raw Colors.* in presentation`
5. Presentation calls cubits only	**644**	188	`students/view/student_screen.dart (16)`
6. Reuse what exists	**~6,900 dup lines**	40+	`attendance/forms/ (10,666 lines, 55% duplicated)`
7. One responsive source	**4 breakpoint sets**	157	`599 vs 600 vs 768 vs 900, all live`
8. Models, not maps	**269 + 486**	108	`269 hand-written parsers, 0 generated; 486 raw maps in domain and presentation`

Read this first. analysis_options.yaml exists in the repo root with every single line commented out, including the include: package:flutter_lints/flutter.yaml directive. flutter analyze currently enforces nothing. Until that file is uncommented and wired into CI, none of the eight mandates below can be held to, because nothing will fail when they are broken. See Section 9.

1. Navigation: GoRouter everywhere, MaterialPageRoute nowhere

1.1 Where we are

go_router ^17.2.3 is a dependency and `lib/src/app_router.dart` defines 56 GoRoute entries across 1,575 lines. That part works. Alongside it, **249 MaterialPageRoute calls survive in 91 files**, up from 108 at the May audit. Normalised for the codebase doubling in size, raw navigation grew 27%. This is the only mandate in this document that got worse per line of code, not just in absolute terms.

File	Raw pushes	Note
`presentation/features/attendance/widgets/attendance_popup.dart`	**26**	Highest single file
`presentation/features/billing/widgets/transactions/transaction_details_screen.dart`	11	Screen living under widgets/
`presentation/features/activities/view/activity_daily_tab.dart`	11	7,065-line file
`presentation/features/students/view/student_screen.dart`	9	
`presentation/features/staff/view/staff_screen.dart`	9	
`presentation/features/attendance/view/staff_attendance_tabbed_view.dart`	8	
`presentation/features/billing/widgets/transactions/transactions_section.dart`	7	
Remaining 84 files	168	1 to 6 each

1.2 The rule

- Every navigation goes through GoRouter: **`context.push(AppRoutes.x)`**, `context.go(...)`, `context.pop()`.
- Route paths and names live in one file as constants. `AppRoutes` does not exist yet and must be created. Today the 56 GoRoute entries carry inline string paths.
- Route arguments are typed. Pass them through `extra` with a declared class, or as path parameters. No `Object?` casts at the receiving end.
- No `Navigator.push`, `Navigator.of(context).push`, or `MaterialPageRoute` anywhere under `lib/presentation`.
- Modals and bottom sheets are exempt. `showDialog` and `showModalBottomSheet` are not route pushes and stay as they are.

1.3 Worked example

From `lib/presentation/features/students/view/student_screen.dart`. This pattern repeats 249 times with different screen classes.

WRONG, as it stands today

```
Navigator.push(
  context,
```

```
  MaterialPageRoute(
    builder: (context) => StudentProfileScreen(
      studentCode: student.code,
    ),
  ),
);
```

RIGHT

```
// core/router/app_routes.dart
abstract final class AppRoutes {
  static const String students     = '/students';
  static const String studentDetail = 'detail';   // child of students
  static String studentDetailPath(String code) => '/students/detail/$code';
}

// core/router/app_router.dart
GoRoute(
  path: AppRoutes.students,
  builder: (_, __) => const StudentScreen(),
  routes: [
    GoRoute(
      path: '${AppRoutes.studentDetail}/:code',
      builder: (_, state) => BlocProvider(
        create: (_) => serviceLocator<StudentProfileCubit>()
          ..load(state.pathParameters['code']!),
        child: const StudentProfileScreen(),
      ),
    ),
  ],
),

// the call site, in the view
context.push(AppRoutes.studentDetailPath(student.code));
```

Why this matters. The 249 raw pushes are why role guards cannot be added centrally, why deep links from a push notification have to re-implement navigation by hand, and why the notification routing layer already contains its own duplicate of the switch-school-then-navigate sequence. Every guard you want either gets pasted into 249 call sites or lives in one router file.

1.4 Migration

This is mechanical and should be done in one sweep, not spread over sprints. The 91 files are known and listed. Order:

- Create `core/router/app_routes.dart` and lift all 56 inline path strings out of `app_router.dart` into constants. No behaviour change.
- Move `lib/src/app_router.dart` to `lib/core/router/app_router.dart` and delete the `lib/src/` folder, which holds nothing else.
- For each screen currently reached by MaterialPageRoute, add a GoRoute if one does not exist, then replace the push. 91 files.
- Add the lint gate so number 250 cannot be written. See Section 9.

1.5 Impact on future maintenance

The kiosk-mode lock is enforced in one place: the redirect callback at app_router.dart:279, which runs before every GoRouter navigation. It does not run before a MaterialPageRoute push, because that push talks to the Navigator directly and never reaches the router at all. A developer who opens a new screen with MaterialPageRoute, copying the file next to theirs, has written a screen that the kiosk lock cannot see, and nothing about the code looks wrong. It compiles, it opens, it just was never in a position to be checked.

The same is true of any guard added to redirect in the future, a role check or a session check included. It protects whatever fraction of navigation goes through the router, and that fraction has to be re-established call site by call site rather than assumed. A bug report that a restricted screen was reachable is, in this codebase, first a question of which of the two navigation paths was used to reach it.

2. State: setState scarcely, ValueNotifier for UI micro-state

2.1 Where we are

1,631 setState calls across 311 StatefulWidgets. The May audit counted 1,403 across 195. The rate per thousand lines improved 36%, which means new code is better behaved, but not a single old call site was cleaned up.

setState itself is fine. The trouble is that it does three unrelated jobs here, and only one of them is a job it should have.

Job setState is doing	How often	Where it belongs
Screen and business state	Lists, filters, form submission, loading flags, API results	**Cubit**
UI micro-state	Expand/collapse, password visibility, tab index, chip selection, hover	**ValueNotifier + ValueListenableBuilder**
Animation and controller ticks	Rare in this codebase (10 AnimationControllers total)	**setState is fine here**

2.2 The rule

- If the state survives a rebuild of the parent, affects more than one widget, or comes from an API, it is **Cubit state**. No exceptions.
- If the state is owned by one widget, never read by anything outside it, and would be thrown away on navigation, use **`ValueNotifier`** with **`ValueListenableBuilder`**. This keeps the widget `StatelessWidget` and rebuilds only the subtree that reads the value.
- A `ValueNotifier` created in a widget must be disposed. If the widget has no other reason to be stateful, hold the notifier as a `final` field on a `StatefulWidget` State and dispose it, or use a small reusable wrapper. Do not create one inside `build()`.
- setState is allowed only for controller-driven ticks and only inside a widget that already needs `State` for a controller lifecycle.

2.3 Worked example: the expand/collapse card, written seven times

Seven separate widgets in this codebase each declare their own `isExpanded` boolean, each make themselves StatefulWidget purely to hold it, and each call setState to flip it.

File and line	Total file size
`core/common/widgets/expandable_info_card.dart:89`	296 lines
`presentation/features/students/widgets/student_card.dart:211`	732 lines
`presentation/features/staff/widgets/staff_card.dart:216`	825 lines
`presentation/features/billing/widgets/billing_student_card.dart:180`	469 lines
`presentation/features/attendance/widgets/student_attendance_card.dart:59`	242 lines
`presentation/features/reports/widgets/report_card.dart:65`	`setState(() => _isExpanded = !_isExpanded)`

File and line	Total file size
presentation/features/reports/widgets/generated_report_card.dart:48	setState(() => _isExpanded = !_isExpanded)

WRONG, the pattern in all seven

```
class StudentCard extends StatefulWidget {   // stateful ONLY for one bool
  ...
}

class _StudentCardState extends State<StudentCard> {
  bool isExpanded = false;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () => setState(() {
        isExpanded = !isExpanded;         // rebuilds the ENTIRE card
      }),
      child: Column(children: [
        _header(),                        // rebuilt for nothing
        _avatar(),                        // rebuilt for nothing
        if (isExpanded) _details(),       // the only thing that changed
      ]),
    );
  }
}
```

RIGHT

```
class StudentCard extends StatelessWidget {   // no State needed
  StudentCard({super.key, required this.student});
  final Student student;

  @override
  Widget build(BuildContext context) {
    return ExpandableInfoCard(              // already exists, see Section 6
      titleText: student.fullName,
      leading: AppAvatar(url: student.photoUrl),
      rows: [
        InfoRowItem(label: AppStrings.classroom, valueText: student.classroomName),
        InfoRowItem(label: AppStrings.dateOfBirth, valueText: student.dobLabel),
      ],
    );
  }
}

// and inside ExpandableInfoCard, ONE implementation of the toggle:
final _expanded = ValueNotifier<bool>(false);

ValueListenableBuilder<bool>(
  valueListenable: _expanded,
  builder: (_, expanded, header) => Column(children: [
    header!,                                // built ONCE, never rebuilt
    if (expanded) _details(),               // only this rebuilds
  ]),
  child: _header(),                         // passed through as `header`
)
```

```
@override
void dispose() { _expanded.dispose(); super.dispose(); }
```

Note the child: parameter. ValueListenableBuilder takes a child that is built once and handed to the builder untouched. That is the actual performance win over setState, and it is the reason this rule exists rather than being style preference. With setState the header, avatar and every const subtree get rebuilt on every tap.

2.4 Worked example: state that should never have been in a widget

From `lib/presentation/features/activities/view/activity_daily_tab.dart`, which holds 68 setState calls in 7,065 lines. Filter selections, loaded feed data and submission flags are all widget-local, so navigating away and back loses them and none of it can be unit tested.

WRONG

```
setState(() {
  _selectedClassroom = classroom;
  _isLoading = true;
});
final result = await _repository.getActivities(classroom.code);
setState(() {
  _activities = result;
  _isLoading  = false;
});
```

RIGHT

```
// view_model/activity_feed_cubit.dart
Future<void> selectClassroom(Classroom classroom) async {
  emit(ActivityFeedLoading(classroom: classroom));
  final result = await _getActivities(classroomCode: classroom.code);
  result.fold(
    (failure) => emit(ActivityFeedError(failure.message)),
    (items)   => emit(ActivityFeedLoaded(classroom: classroom, items: items)),
  );
}

// view/activity_daily_tab.dart
BlocBuilder<ActivityFeedCubit, ActivityFeedState>(
  builder: (context, state) => switch (state) {
    ActivityFeedLoading()                   => const ActivityFeedSkeleton(),
    ActivityFeedLoaded(items: final items)  => ActivityFeedList(items: items),
    ActivityFeedError(message: final m)     => AppErrorView(message: m),
    _                                       => const SizedBox.shrink(),
  },
)
```

2.5 Migration

Do not attempt to remove 1,631 setState calls in one pass. Ratchet instead:

- Ban new ones. A CI grep on the diff is enough: any added `setState` under `lib/presentation` needs a reviewer to sign off in the PR.
- Sweep the 45 single-boolean toggles first. They are identified, they are one-line changes each, and several of them disappear entirely when the duplicate cards collapse into `ExpandableInfoCard` (Section 6).
- Any file you open for a feature change must leave with fewer setState calls than it had. This is the only rule that will move the top ten files.
- Track the raw count in CI so the number is visible and cannot silently climb back.

2.6 Impact on future maintenance

setState marks the whole State object dirty, so build() reruns for everything under it, not just the part that changed. A card that shows a photo, a name and a couple of badges alongside an expand toggle rebuilds the photo and the badges every time the toggle is tapped, because they live in the same build() as the part that actually changed. In a list of twenty such cards this is the drop in frame rate a developer sees when scrolling and tapping at the same time, and the natural place to look for it is the toggle logic, when the actual cost is everything else in that build() coming along for the ride.

ValueNotifier paired with ValueListenableBuilder fixes this by construction rather than by care. The header and photo are built once and passed in through the child: parameter; only the small widget that reads the notifier rebuilds when it changes. The same card, converted, does the same thing on tap while touching a fraction of what it used to.

3. Folder structure, followed strictly

The target structure below is reproduced unchanged from the May 2026 standard. It has not been amended. One addition is proposed at the end of this section, for Section 7.

3.1 The canonical structure

```
lib/
├── core/
│   ├── constants/              # All app-wide constants
│   │   ├── app_strings.dart       # All UI strings (no literals in view files)
│   │   ├── app_sizes.dart         # Spacing, radius, icon sizes
│   │   ├── app_durations.dart     # Animation / debounce durations
│   │   ├── storage_keys.dart      # All SharedPrefs / SecureStorage keys
│   │   └── app_modules.dart       # Module name constants for AuthorityService
│   ├── di/
│   │   ├── di.dart                # Root: calls all feature modules
│   │   └── modules/               # One file per feature (auth_module, billing_module...)
│   ├── extensions/
│   │   ├── string_ext.dart
│   │   ├── date_ext.dart
│   │   ├── int_ext.dart
│   │   ├── list_ext.dart
│   │   └── context_ext.dart       # theme, mediaQuery, session, navigation helpers
│   ├── router/
│   │   ├── app_router.dart        # GoRouter config, guards
│   │   ├── app_routes.dart        # Route name constants
│   │   └── guards/                # auth_guard, role_guard
│   ├── services/               # All 3rd-party package abstractions
│   │   ├── storage/               # StorageService (abstract) + impls
│   │   ├── network/               # NetworkService (abstract) + DioNetworkService
│   │   ├── logger/                # LoggerService (abstract) + impl
│   │   ├── crash/                 # CrashService (abstract) + FirebaseCrash/Sentry
│   │   ├── push_notification/     # PushNotificationService (abstract)
│   │   ├── device_info/           # DeviceInfoService (abstract)
│   │   ├── asset_picker/          # AssetPickerService (abstract)
│   │   ├── analytics/             # AnalyticsService (abstract)
│   │   └── authority/             # AuthorityService (abstract) + impl
│   ├── startup/
│   │   └── app_startup.dart       # AppStartup engine (init, verify, bootstrap)
│   ├── theme/
│   │   ├── app_colors.dart        # All colour tokens
│   │   ├── app_text_styles.dart   # Typography scale
│   │   ├── app_theme.dart         # ThemeData (light + dark)
│   │   └── app_decorations.dart   # Shared BoxDecoration, InputDecoration
│   ├── validators/
│   │   └── app_validators.dart    # All form validators (composable)
│   └── widgets/                # Widgets used in 2+ features
│       ├── app_button.dart
│       ├── app_text_field.dart
│       ├── app_scaffold.dart
│       ├── app_loader.dart        # Shimmer skeleton
│       ├── app_avatar.dart
│       ├── app_empty_state.dart
│       ├── app_error_view.dart
│       └── media_viewer.dart      # Image / video / pdf viewer
│
├── data/
│   └── features/
```

```
│       └── <feature_name>/         # snake_case, e.g. auth, attendance, billing
│           ├── datasource/            # remote_datasource.dart + local_datasource.dart
│           ├── models/                # DTOs (JSON to Dart), NO Flutter imports
│           ├── mappers/               # model_to_entity mapper
│           └── repository/            # _impl.dart (implements domain contract)
│
├── domain/
│   └── features/
│       └── <feature_name>/
│           ├── entities/              # Pure Dart. NO fromJson/toJson here.
│           ├── repositories/          # Abstract contracts only
│           └── usecases/              # One class, one public method: call()
│
├── presentation/
│   └── features/
│       └── <feature_name>/         # MVVM per feature
│           ├── model/                 # Screen params / UI config objects
│           ├── view/                  # Screen files. ZERO logic. ZERO API calls.
│           ├── view_model/            # Cubit + State files
│           └── widgets/               # This feature only. Shared goes to core/widgets/
│
├── main.dart           # ~30 lines. runApp() only.
└── .env                # Project root. Never committed to git.
```

3.2 Where the current tree diverges

Nineteen features live under `lib/presentation/features/` and no two of them agree on their own layout. The same concept has up to three names.

Concept	Required name	Also in use	Where
ViewModel layer	`view_model/`	`cubit/` `viewmodels/`	17 features use cubit/; dashboard uses viewmodels/; onboarding and user-profile use view_model/
Screens	`view/`	`views/`	classroom is the only views/
Feature widgets	`widgets/`	`widget/`	auth is the only widget/
Feature helpers	`no such folder`	`util/` `utils/`	util/ in 4 features, utils/ in 6. Pure logic belongs in domain or core/extensions.
Screen params	`model/`	`mostly absent`	Only scheduling has one. Billing has two under widgets/.

3.3 Specific files, and where they go

Not an exhaustive list. These are the cases where the current location is clearly wrong and the correct one is clear.

Structural, do these first

Currently at	Move to	
lib/src/app_router.dart	lib/core/router/app_router.dart	1 file
lib/bootstrap/app_bootstrap.dart	lib/core/startup/app_startup.dart	1 file
lib/app/di/di.dart (2,492 lines, 420 registrations)	lib/core/di/di.dart + lib/core/di/modules/<feature>_module.dart	19 files
lib/app/theme/*	lib/core/theme/* (see Section 4 for the renames)	5 files
lib/core/util/storage_keys.dart	lib/core/constants/storage_keys.dart	1 file
lib/core/util/api_paths.dart, api_constants.dart, api_paths_uat.dart, api_constants_uat.dart	lib/core/constants/api_paths.dart and lib/core/config/app_config.dart. The _uat pair should not exist as separate files, the environment picks the base URL.	4 to 2
lib/core/util/logger_service.dart, server_time_service.dart	lib/core/services/logger/, lib/core/services/time/	2 files
lib/core/util/{staff,guardian,school}_code_resolver.dart, school_timezone_resolver.dart, attendance_cross_class_resolver.dart	These are business rules. They belong in domain/features/<x>/usecases/ or as domain services, not in a util folder.	5 files
lib/core/util/{attendance,billing,notification,school_switch}_events.dart	lib/core/events/ as a single named bus, or delete in favour of cubit-to-cubit subscription.	4 files
lib/core/common/widgets/expandable_info_card.dart	lib/core/widgets/expandable_info_card.dart. There is no reason for both core/common/widgets/ (1 file) and shared/common/widgets/ (85 files) to exist.	1 file
lib/shared/common/widgets/ (85 files)	lib/core/widgets/	85 files
lib/presentation/features/communication/presentation/messaging/...	Flatten. A presentation folder nested inside the presentation layer means every path in that feature reads presentation twice.	~40 files
lib/presentation/features/dashboard/services/dashboard_cubit_factory.dart	Cubit construction belongs in DI, not in a services folder inside presentation.	1 file
lib/presentation/features/students/module_access.dart	Loose file at a feature root. Move under domain if it is a rule, or view_model/ if it is UI gating.	1 file
lib/presentation/features/billing/widgets/models/ and billing/widgets/general_ledger/models/	data/features/billing/models/ for DTOs, domain/features/billing/entities/ for entities. Named in the May audit as A-03 and unmoved since.	2 folders
lib/presentation/features/billing/widgets/transactions/transaction_details_screen.dart	billing/view/. A file ending in _screen is not a widget.	several

Naming, fix in the same sweep

Currently	Should be	
presentation/features/dashboard/view/QR_Code_scanner.dart	qr_code_scanner.dart	
scheduleDataModel.dart	schedule_data_model.dart, and it belongs in data/features/scheduling/models/	
attendance/forms/student_attendance_forms/student_signIn_out_button.dart	student_sign_in_out_button.dart	

Currently	Should be	
`classroom/views/classroom_tab_cards/Toggel-Service/`	Directory name is kebab-case and contains a typo for "toggle". Rename to toggle_service/ and move its napping_students_cubit.dart into the feature view_model/.	
`.../Toggel-Service/toogel-api-service.dart`	Second spelling of the same typo, in the same path. This is a datasource and belongs in data/features/classroom/datasource/.	
`presentation/features/billing/view/Accounting/`	`accounting/`	
`presentation/features/user-profile/`	`user_profile/` (snake_case, not kebab)	

3.4 Imports: the reason the rename is not cosmetic

There are 2,124 relative imports in lib/, 923 of them going up three or more levels and 891 going up four or more. Beyond being unreadable, a specific class of them is actively broken.

Dart resolves a relative import against the `package:` URI of the importing file, and per RFC 3986 it silently discards excess `..` segments once it reaches the package root. So an import with one `..` too many still compiles. It stops compiling the moment the same file is reached through a `file:` URI, which is exactly what a test doing `import '../../../../../lib/...'` does.

File with an excess ".." segment	Consequence today
`lib/shared/common/widgets/selection_list_item.dart:2`	Compiles in the app. Would break any test that reaches it by relative path.
`lib/domain/features/messaging/usecases/get_feature_flag_usecase.dart:2`	Breaks feature_flag_cubit_test.dart, which currently fails to load at all.
`lib/domain/features/messaging/usecases/set_feature_flag_usecase.dart:2`	Same.

The rule. All cross-folder imports use package:wowcare/... form. Relative imports are allowed only for files in the same folder. Tests import lib code as package:wowcare/..., never as ../../../lib/.... This is a single codemod and it removes the entire class of failure above.

3.5 One addition to the target structure

Section 7 needs a home. Add one folder to `core/`:

```
│   ├── responsive/
│   │   ├── app_breakpoints.dart    # THE breakpoint values. Nothing else defines them.
│   │   ├── responsive_scope.dart   # Single LayoutBuilder, InheritedWidget below it
│   │   └── responsive_ext.dart     # context.isTablet, context.gutter, context.columns
```

3.6 Impact on future maintenance

The gates proposed in Section 9 are path-based checks, and so is most tooling that scopes itself by folder: an IDE snippet, a saved search, a CI grep. A check written against lib/presentation/features/*/view/ does not see billing, which keeps its screens under view/Accounting/, or classroom, which uses views/ instead. Those files still compile and still ship. The check

simply does not run against them, and nothing reports that it skipped anything, because a rule that never matches a path does not fail, it stays silent.

A developer adding a feature has to learn which of the three spellings their feature already committed to before their file lands in the one a later rule, search, or reviewer will actually look at. A developer fixing a bug that spans features has to check the folder name each feature uses individually, because a single search across view/ misses whatever is filed under views/.

4. Centralized theming, constants, strings, colors, fonts and text styles

4.1 Where we are

This mandate has a different character to the others. The infrastructure is largely **already built**. It is simply not called. Some of it has never been called even once.

Token that exists	Uses	Bypassed by	Verdict
`AppColors (242 lines)`	2,142	**4,635**	3,479 raw Colors.* plus 1,156 raw Color(0x..). Bypassed 2 to 1.
`lightTheme / darkTheme textTheme`	**11**	**1,501**	A full TextTheme is defined and wired into ThemeCubit. Ignored 136 to 1.
`Fonts.manrope`	44	**882**	fontFamily: 'Manrope' written by hand 882 times. Bypassed 20 to 1.
`ContainerDecorations (168 lines)`	**1**	everything	Seven BoxDecorations and an InputDecoration, one external caller.
`ButtonStyles`	**0**	everything	Never called. Dead since it was written.
`GradientBorderContainer`	**0**	everything	Never called. Dead since it was written.

And the pieces that do not exist at all:

- **`app_strings.dart`**. 757 Text() widgets with inline literals across 233 files, 1,082 literal label/title/hint/tooltip values, 1,136 SnackBar call sites with inline copy.
- **`app_sizes.dart`**. 1,570 magic EdgeInsets and 2,407 magic SizedBox dimensions in the presentation layer.
- **`app_text_styles.dart`**. The TextTheme in app_themes.dart covers five slots. Nothing names the other styles the app actually uses.
- **`app_durations.dart`**. Animation and debounce durations are inline everywhere.

4.2 The specific problem with dark mode

A complete `darkTheme` exists at `lib/core/util/app_themes.dart:41` and `ThemeCubit.toggleTheme()` will switch to it. It cannot work. 3,479 hardcoded `Colors.*` references and 1,156 hardcoded hex values do not respond to a theme change, so flipping the toggle produces a screen that is half dark and half white text on white.

Separately, `ThemeCubit` compares `state == lightTheme` to decide which way to toggle. ThemeData equality is structural and fragile. Hold a `ThemeMode` enum in state instead and resolve the ThemeData at the MaterialApp.

4.3 The rule

- No `Color(0x...)`, no `Colors.blue` style references outside `core/theme/app_colors.dart`. Every colour resolves through `AppColors` or `Theme.of(context).colorScheme`.
- No inline `TextStyle(`. Use `Theme.of(context).textTheme.<slot>` or a named style from `AppTextStyles`. Local variation uses `.copyWith()` on a named style, never a fresh TextStyle.

- No `fontFamily: 'Manrope'`. The family is set once on ThemeData and inherited.
- No user-visible string literal in a view file. Everything goes through `AppStrings`. This also makes localisation a later mechanical job rather than a rewrite.
- No raw spacing numbers. `AppSizes.md`, not `EdgeInsets.all(16)`.
- Delete `ButtonStyles` and `GradientBorderContainer` or start using them. Zero-caller theme helpers are worse than none, because the next person writes a third version rather than trusting them.

4.4 Worked example

Composite of patterns from `billing/widgets/transactions/view_receipt_screen.dart` (27 inline TextStyles) and `dashboard/view/widgets/my_day_card.dart` (18).

WRONG

```
Container(
  padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
  decoration: BoxDecoration(
    color: const Color(0xFFF7F8FA),
    borderRadius: BorderRadius.circular(12),
    border: Border.all(color: Colors.grey.shade300),
  ),
  child: Text(
    'Payment received',
    style: TextStyle(
      fontSize: 16,
      fontWeight: FontWeight.w600,
      fontFamily: 'Manrope',
      color: Colors.black87,
    ),
  ),
)
```

RIGHT

```
// core/constants/app_sizes.dart
abstract final class AppSizes {
  static const double xs = 4, sm = 8, md = 12, lg = 16, xl = 24;
  static const double radiusCard = 12;
  static const EdgeInsets cardPadding =
      EdgeInsets.symmetric(horizontal: lg, vertical: md);
}

// core/constants/app_strings.dart
abstract final class AppStrings {
  static const String paymentReceived = 'Payment received';
}

// core/theme/app_decorations.dart
abstract final class AppDecorations {
  static BoxDecoration card(BuildContext context) => BoxDecoration(
    color: Theme.of(context).colorScheme.surfaceContainerLow,
    borderRadius: BorderRadius.circular(AppSizes.radiusCard),
    border: Border.all(color: AppColors.dividerColor),
  );
```

```
}

// the call site
Container(
  padding: AppSizes.cardPadding,
  decoration: AppDecorations.card(context),
  child: Text(
    AppStrings.paymentReceived,
    style: Theme.of(context).textTheme.titleMedium,   // family + colour inherited
  ),
)
```

4.5 Migration

- Move `lib/app/theme/*` and `lib/core/util/app_themes.dart` into `lib/core/theme/` as `app_colors.dart`, `app_theme.dart`, `app_text_styles.dart`, `app_decorations.dart`. Rename `ContainerDecorations.decoration0` through `decoration6` to names that say what they are.
- Create `app_strings.dart` and `app_sizes.dart`. Extract the 757 Text() literals and the top twenty spacing values first, then the long tail.
- The 882 hardcoded `fontFamily: 'Manrope'` lines are a single find and delete. The family is already on ThemeData.
- Colours and TextStyles are the long pole. Do them by feature, not globally, and take the dark theme seriously only after that sweep completes.

4.6 Impact on future maintenance

Theme.of(context).textTheme.bodyMedium.copyWith(fontWeight: FontWeight.w600) keeps every property the theme already defines and changes only the one that is named. A hand-written TextStyle(fontSize: 14, fontFamily: 'Manrope', color: AppColors.primaryColor) restates all of it, so it tracks nothing. If the body text style later needs a letter-spacing adjustment for legibility, or the brand switches font, the 1,501 hand-written styles do not receive it, they have to be found and edited one at a time.

Colour has the sharper version of the same problem. lightTheme and darkTheme both exist and ThemeCubit already switches between them, so a developer testing the toggle reasonably expects it to work. What they get is a screen where the tokenised parts change and the 4,635 hardcoded Colors and hex values do not, and the fix is not a theme edit but a manual pass over that one screen to find which of its colours were never tokens in the first place.

5. Presentation talks to cubits only

5.1 Where we are

Real progress here. Direct data-layer references in the presentation layer went from 574 to 144, a 75% reduction, and the two worst patterns are gone: **zero** direct RepositoryImpl or DataSourceImpl instantiations in a widget, and **zero** casts of a repository interface to its concrete type. 91 of 115 cubits go through use cases.

What has not moved is the service locator. **644 serviceLocator<T>() calls sit in lib/presentation across 188 files**, 121 of those files being view files. This was C-06 in the May audit and it is untouched.

File	Calls	What it resolves
`students/view/student_screen.dart`	**16**	Cubits and use cases
`attendance/view/screens/student_attendance_daily_view.dart`	**16**	Cubits and use cases
`classroom/views/screens/classroom_info_screen.dart`	12	
`attendance/view/screens/staff_attendance_daily_view.dart`	12	
`students/view/student_profile_screen.dart`	11	StudentRemoteDataSource, directly
`attendance/forms/staff_attendance_forms/staff_sign_in_form.dart`	11	
`dashboard/view/widgets/my_day_card.dart`	10	A widget, not even a screen

5.2 The three violations, in order of severity

a. A view calling a datasource

The presentation layer reaching past the repository and the use case, straight into data.

```
// presentation/features/students/view/student_profile_screen.dart:203, 248, 420
final settings = await serviceLocator<StudentRemoteDataSource>()
    .getPrivacySettings(studentCode);

// presentation/features/attendance/view/student_attendance_profile.dart:599
final profile = await serviceLocator<StudentRemoteDataSource>()
    .getStudentProfile(code);

// presentation/features/auth/view/sign_in_screen.dart:80, 140
final localDataSource = serviceLocator<AuthLocalDataSource>();
final credentials     = await localDataSource.getSavedCredentials();
await localDataSource.saveCredentials(username, password, rememberMe);
```

sign_in_screen.dart is the same file, doing the same thing, at nearly the same line numbers as the May 2026 audit recorded under C-06. It was cited by name fifteen months of commits ago and has not been touched. Handling credentials from a view file is also the highest-risk instance of this pattern in the codebase.

b. A view handing the raw HTTP client to something

```
// presentation/features/activities/view/activity_daily_tab.dart:2632
apiClient: serviceLocator<ApiClient>(),
```

c. A view constructing its own cubit

144 call sites do `serviceLocator<SomeCubit>()` inside initState or build rather than receiving the cubit from a BlocProvider set up by the router. The cubit lifecycle then belongs to nobody, the same cubit type can have several live instances, and state is not preserved correctly across navigation.

```
// presentation/features/activities/view/activity_daily_tab.dart:2562
_moderationCubit = serviceLocator<ActivityModerationCubit>();
_storageService  = serviceLocator<StorageService>();
```

5.3 The rule

- A view file may reference exactly three things from outside itself: its own Cubit via `context.read<T>()` / `BlocBuilder`, widgets, and constants. Nothing else.
- No `serviceLocator<T>()` in `lib/presentation`. Cubits are created in the router, in the GoRoute builder, wrapped in a BlocProvider. That is the only place the service locator appears on the presentation side.
- No DataSource, Repository, ApiClient, Dio or SharedPreferences type may be named in a view file.
- Cubits call use cases. The 36 cubits still importing a repository directly and the 12 still importing a datasource get migrated as their features are touched.
- No `FutureBuilder` or `StreamBuilder` in a view over data the cubit already owns. There are 25 of these and each one is a second source of truth for state the cubit is already tracking.

5.4 Worked example

WRONG, from student_profile_screen.dart

```
class _StudentProfileScreenState extends State<StudentProfileScreen> {
  bool _isPrivate = false;

  @override
  void initState() {
    super.initState();
    _loadSettings();
  }

  Future<void> _loadSettings() async {
    final settings = await serviceLocator<StudentRemoteDataSource>()
        .getPrivacySettings(widget.studentCode);
    if (!mounted) return;
    setState(() => _isPrivate = settings.isPrivate);
  }
}
```

RIGHT

```
// domain/features/student/usecases/get_privacy_settings_usecase.dart
class GetPrivacySettingsUseCase {
  const GetPrivacySettingsUseCase(this._repository);
  final StudentRepository _repository;

  Future<Either<AppFailure, PrivacySettings>> call(String studentCode) =>
      _repository.getPrivacySettings(studentCode);
}

// presentation/features/students/view_model/student_profile_cubit.dart
Future<void> loadPrivacySettings(String studentCode) async {
  final result = await _getPrivacySettings(studentCode);
  if (isClosed) return;
  result.fold(
    (f) => emit(state.copyWith(error: f.message)),
    (s) => emit(state.copyWith(isPrivate: s.isPrivate)),
  );
}

// core/router/app_router.dart  <- the ONLY serviceLocator on this path
GoRoute(
  path: '/students/detail/:code',
  builder: (_, state) {
    final code = state.pathParameters['code']!;
    return BlocProvider(
      create: (_) => serviceLocator<StudentProfileCubit>()
        ..loadPrivacySettings(code),
      child: const StudentProfileScreen(),
    );
  },
),

// presentation/features/students/view/student_profile_screen.dart
class StudentProfileScreen extends StatelessWidget {          // no State at all
  const StudentProfileScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<StudentProfileCubit, StudentProfileState>(
      builder: (_, state) => PrivacyToggle(value: state.isPrivate),
    );
  }
}
```

One caution on the router change. Putting ..load() inside a GoRoute builder or pageBuilder makes it re-run on every navigation to that route, including back navigation. Use the builder for construction and trigger the load from the cubit constructor or a single BlocProvider create, so the same screen does not fire its API two or three times. This has already caused duplicate-call bugs in this codebase.

5.5 Impact on future maintenance

A cubit created through BlocProvider is closed automatically when the provider leaves the widget tree. A cubit resolved with serviceLocator<XCubit>() inside initState is not owned by any BlocProvider, so nothing closes it unless the view remembers to do so itself, and 56 of the 98 files that build a cubit this way never call .close() anywhere. Because most feature cubits are registered as a factory, a fresh instance is created each time the screen opens, and left unclosed,

whatever it was waiting on when the screen was left keeps running in the background and still calls emit when it finishes, on a cubit nothing is listening to any more.

The other side of the same gap is direct data access. student_profile_screen.dart calls StudentRemoteDataSource itself at lines 203, 248 and 420 rather than through its cubit. A bug affecting the data on that screen can originate in the cubit or in the view, and establishing which one it is is the first step, before any actual fix.

6. Pure functions and reusable widgets, actually used

6.1 Where we are

There are 665 widget classes declared inside `lib/presentation` and an 85-file shared widget library. The shared library is not the problem. The problem is that when a developer needs a card, a list tile or a bottom sheet, the cheapest path is to write a new one, and nothing stops them.

Class name	Copies	Locations
`StaffListItem`	6	4 in attendance/forms/staff_attendance_forms/, 1 in classroom/forms/assign_staff_to_classroom_form.dart, 1 in staff/widgets/staff_list_item.dart
`StudentListItem`	5	4 in attendance/forms/student_attendance_forms/, 1 in classroom/forms/assign_student_to_classroom_form.dart
`RemoveStudentTile`	5	4 live copies in attendance/forms/, plus shared/common/widgets/remove_student_tile.dart which is entirely commented out
`StudentAttendanceCard`	3	attendance/view/screens/widgets/, attendance/widgets/, dashboard/view/widgets/. Three different StatefulWidgets, one name.
`DynamicModalBottomSheet`	3	All three in shared/common/widgets/reusable_modal_bottom_sheet.dart. One live at line 58, two commented out at lines 449 and 795.
`_Chip`, `_EmptyState`, `_DialogField`, `_DialogDropdown`, `_SectionLabel`, `_ScheduleChip`, `_DateRow`	3 to 4 each	Private duplicates across unrelated features

6.2 The clearest case: a reusable widget with zero callers

`lib/core/common/widgets/expandable_info_card.dart` is 296 lines. It exposes leading, collapsedPrefix, collapsedPrefixSpacing, titleText, titleStyle, subtitleText, subtitleStyle, collapsedHeight, headerPadding and a typed `InfoRowItem` list. Someone built a proper, general expandable card.

It is referenced **zero times outside its own file**. Meanwhile four feature cards totalling 2,268 lines hand-roll the same expand and collapse behaviour, each with its own setState (see Section 2.3).

Exists and is unused	Written instead, by hand

`core/common/widgets/expandable_info_card.dart`
296 lines, 0 external callers
	`students/widgets/student_card.dart (732)`
	`staff/widgets/staff_card.dart (825)`
	`billing/widgets/billing_student_card.dart (469)`
	`attendance/widgets/student_attendance_card.dart (242)`
	2,268 lines total

6.3 The largest case: the attendance form family

`lib/presentation/features/attendance/forms/` holds 23 files and 10,666 lines. Eight of them are selection forms of 444 to 512 lines each, and they are copies of one another.

Measured directly: `student_selection_sign_in_form.dart` and `student_selection_mark_absent_form.dart` share **262 identical lines out of roughly 480**. That is 55% of two files that are supposed to be different features.

The eight selection forms:

Student side	Staff side
`student_selection_sign_in_form.dart` (512)	`staff_selection_sign_in_form.dart` (458)
`student_selection_sign_out_form.dart` (475)	`staff_selection_sign_out_form.dart` (455)
`student_selection_transfer_form.dart` (459)	`staff_selection_transfer_form.dart` (455)
`student_selection_mark_absent_form.dart` (455)	`staff_selection_mark_absent_form.dart` (444)

RIGHT

```
// ONE form, parameterised by what actually differs.

enum AttendanceAction { signIn, signOut, transfer, markAbsent }

class AttendanceSelectionForm<T extends Attendee> extends StatelessWidget {
  const AttendanceSelectionForm({
    super.key,
    required this.action,
    required this.candidates,
    required this.onSubmit,
  });

  final AttendanceAction action;
  final List<T> candidates;
  final ValueChanged<List<T>> onSubmit;

  @override
  Widget build(BuildContext context) => Column(children: [
    AppSearchField(hint: AppStrings.searchFor(action)),
    Expanded(child: ListView.builder(
      itemCount: candidates.length,
      itemBuilder: (_, i) => AttendeeListItem(attendee: candidates[i]),
    )),
    AppButton(label: AppStrings.actionLabel(action), onPressed: ...),
  ]);
}

// AttendeeListItem replaces StaffListItem x6 and StudentListItem x5.
// The action-specific copy lives in AppStrings, not in eight files.
```

Realistic outcome: roughly 3,700 lines of selection forms collapse to about 700, and eleven duplicate list-item classes become one.

6.4 Pure functions

Two patterns keep logic where it cannot be tested.

a. Business calculation inside a view

70 date and duration computations (`.difference()`, `.inDays`, `.inMinutes`, `.inHours`) run inside view files. Each one is a pure function trapped in a widget.

```
// WRONG, inside build()
final months = DateTime.now().difference(student.dateOfBirth).inDays ~/ 30;
Text('$months months old')

// RIGHT, on the entity where it can be unit tested in isolation
// domain/features/student/entities/student.dart
int get ageInMonths => DateTime.now().difference(dateOfBirth).inDays ~/ 30;

// the view
Text(AppStrings.ageInMonths(student.ageInMonths))
```

b. Widget-returning private methods

There are **467 Widget _buildXxx() methods** in the codebase. A method returning a Widget cannot be const, cannot be given a key, and rebuilds with its whole enclosing widget. A private widget class in the same file costs the same to write and does none of that.

```
// WRONG
Widget _buildHeader() => Row(children: [...]);

// RIGHT, in the same file, private to it
class _Header extends StatelessWidget {
  const _Header({required this.title});
  final String title;

  @override
  Widget build(BuildContext context) => Row(children: [...]);
}
```

6.5 The rule

- Before writing a widget, search `core/widgets/` for it. If something close exists, extend it with a parameter rather than copying it.
- A widget used by two or more features moves to `core/widgets/` immediately, at the moment the second use appears, not later.
- No two classes in the codebase share a name. If `StudentListItem` exists, the second one is either the same widget or it is misnamed.
- Widget-returning private methods are replaced by private widget classes.
- Pure logic (dates, formatting, derived values, validation) lives on the entity or in `core/extensions/`, never inside build().
- Commented-out code is deleted, not kept. `reusable_modal_bottom_sheet.dart` holding two dead copies of its own class, and `remove_student_tile.dart` being commented out in full while four copies live in feature files, are both direct consequences of not deleting.

Dead code, measured. 115 of 1,389 files in lib/ are unreachable from main.dart, which is 8.3% of the codebase. Roughly 3,797 lines of commented-out code sit inside the reachable remainder. One dead island (feature_flag_cubit.dart and its two use cases) does not even compile, which is why feature_flag_cubit_test.dart currently fails to load. Deleting unreachable files is safe, mechanical, and should happen before any of the refactors above, so nobody spends time modernising code that nothing calls.

6.6 Impact on future maintenance

The six classes named StaffListItem are not interchangeable. Five take a Staff or StaffEntity object and are built without a const constructor. The sixth, in staff/widgets/staff_list_item.dart, takes plain fullName and role strings and is const. A developer who searches for StaffListItem and reasonably reuses the one that is easiest to find can hit a constructor mismatch the moment they try to pass it the object they actually have, because the name is shared and the shape is not.

The signedInBy defect described in Section 8 is what this looks like once it reaches production rather than compile time: three copies of one field read, two updated for a backend alias and one left behind, so two screens in the same feature now answer the same question differently. Reuse is the fix for both. It only works once a developer has read enough of the existing copies to know which one, if any, is safe to extend.

7. Responsiveness: one LayoutBuilder, tablet and mobile everywhere

7.1 Where we are

There is already one good idea in place. `lib/app/app_sizer_init.dart` sets the sizer Device globals during the build phase by reading the ambient MediaQuery, specifically so that MaterialApp.router does not have to be nested inside a LayoutBuilder. The reasoning in its doc comment is correct and the file should survive the refactor.

Everything around it is the problem. **Four different definitions of "tablet" are live at the same time**, in four different files, and they disagree.

Definition site	Tablet at	Large at	Importers
`app/app_sizer_init.dart`	**599**	-	sizer Device globals
`shared/utils/responsive_sizing.dart:7`	**>= 600**	-	27 files
`shared/utils/responsive_helper.dart:8`	**>= 768**	>= 1024	116 files
`shared/common/widgets/calendar_responsive_helper.dart:8`	**> 600**	> 900	6 files
`shared/common/widgets/custom_app_bar.dart:815`	> 600	-	inline, local
`students/view/student_screen.dart:1514, 1547, 1608`	>= 600	>= 900	inline, local
`activities/view/activity_feed_screen.dart:271, 341`	>= 600	>= 900	inline, local

What this means concretely. A device 700 points wide is a tablet according to responsive_sizing, calendar_responsive_helper, custom_app_bar and every inline check, and a phone according to responsive_helper, which 116 files import. On one screen the app bar lays out for tablet while the body lays out for phone. Nobody can reason about a layout bug without first working out which helper that particular widget happened to import.

Three further problems:

- **`sizer`** is a dependency but is imported by 3 files and there is exactly **one** usage of its `.h/.w/.sp` extensions in the entire codebase. It is paying its weight for nothing.
- **49 LayoutBuilder instances** scattered through feature files, plus 261 MediaQuery calls across 157 files. Each one is an independent decision about what the current size means.
- **2,367 hardcoded pixel width and height values** in the presentation and shared layers. These do not respond to screen size at all, which is the actual reason tablet layouts look wrong rather than merely inconsistent.

7.2 The rule

- Breakpoints are defined once, in `core/responsive/app_breakpoints.dart`. No other file contains a screen-width comparison against a literal number.
- One LayoutBuilder, at the top of the widget tree, inside `ResponsiveScope`. It publishes the resolved size class through an InheritedWidget. Feature code reads `context.screenSize`, it does not measure.

- A LayoutBuilder in a feature file is allowed only when the widget must respond to **its own box**, not the screen. A card that reflows inside a narrow column is a legitimate use. There are three of these today (the two attendance cards checking `constraints.maxWidth < 300` and the dashboard gutter checks) and they should stay.
- Every screen must be checked at 375 (phone), 768 (tablet portrait) and 1024 (tablet landscape) before merge. Not "should look fine".
- Delete `sizer` from pubspec once `app_sizer_init.dart` is replaced. One usage does not justify a dependency.
- No hardcoded pixel dimension for anything that should scale. Fixed sizes are for icons and borders. Content uses Flexible, Expanded, AspectRatio, or a token from `AppSizes`.

7.3 The single source

```
// core/responsive/app_breakpoints.dart
enum ScreenSize { compact, medium, expanded }   // phone, tablet, large tablet

abstract final class AppBreakpoints {
  static const double medium   = 600;   // >= 600 is a tablet
  static const double expanded = 1024;  // >= 1024 is a large tablet

  static ScreenSize of(double width) =>
      width >= expanded ? ScreenSize.expanded
      : width >= medium ? ScreenSize.medium
      : ScreenSize.compact;
}

// core/responsive/responsive_scope.dart
class ResponsiveScope extends StatelessWidget {
  const ResponsiveScope({super.key, required this.child});
  final Widget child;

  @override
  Widget build(BuildContext context) => LayoutBuilder(   // THE LayoutBuilder
    builder: (context, constraints) => _ResponsiveData(
      screenSize: AppBreakpoints.of(constraints.maxWidth),
      width: constraints.maxWidth,
      child: child,
    ),
  );
}

class _ResponsiveData extends InheritedWidget {
  const _ResponsiveData({
    required this.screenSize, required this.width, required super.child,
  });
  final ScreenSize screenSize;
  final double width;

  @override
  bool updateShouldNotify(_ResponsiveData old) =>
      old.screenSize != screenSize;   // rebuild only when the CLASS changes
}

// core/responsive/responsive_ext.dart
extension ResponsiveContext on BuildContext {
  ScreenSize get screenSize =>
      dependOnInheritedWidgetOfExactType<_ResponsiveData>()!.screenSize;
```

```
  bool get isTablet => screenSize != ScreenSize.compact;

  T responsive<T>({required T compact, T? medium, T? expanded}) =>
      switch (screenSize) {
        ScreenSize.expanded => expanded ?? medium ?? compact,
        ScreenSize.medium   => medium ?? compact,
        ScreenSize.compact  => compact,
      };
}
```

Why updateShouldNotify compares the size class, not the width. If the InheritedWidget notifies on every pixel of width change, every dependent widget in the app rebuilds during a rotation animation or a soft-keyboard transition. Comparing the enum means dependants rebuild exactly once, when the layout genuinely crosses a breakpoint.

7.4 Worked example

WRONG, the current pattern in student_screen.dart

```
// student_screen.dart:1514
final isTablet      = MediaQuery.of(context).size.width >= 600;
final isLargeTablet = MediaQuery.of(context).size.width >= 900;

// ...94 lines later, a different rule, same file
LayoutBuilder(builder: (context, constraints) {
  final bool isTablet = constraints.maxWidth >= 600;
  ...
});

SizedBox(width: 320, child: ...)          // does not scale at all
```

RIGHT

```
// app.dart, once
MaterialApp.router(
  builder: (context, child) => ResponsiveScope(child: child!),
  ...
)

// student_screen.dart, and every other screen
GridView.builder(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: context.responsive(compact: 1, medium: 2, expanded: 3),
    mainAxisSpacing: AppSizes.md,
  ),
  itemBuilder: (_, i) => StudentCard(student: students[i]),
)

// and where a fixed width was hardcoded
ConstrainedBox(
  constraints: BoxConstraints(
    maxWidth: context.responsive(compact: double.infinity, medium: 420),
  ),
  child: ...,
)
```

7.5 Migration

- Create the three files in `core/responsive/` and mount ResponsiveScope in app.dart. Nothing breaks, nothing changes yet.
- Pick the breakpoints and settle the argument once. 600 and 1024 match the Material window size classes and match what the majority of the existing inline checks already assume.
- Redirect the three helper files to delegate to `AppBreakpoints` instead of deleting them outright. That fixes the disagreement across 149 importing files in one commit, without touching 149 files.
- Then delete the helpers, migrate their callers to `context.responsive(...)`, and remove `sizer` from pubspec.
- The 2,367 hardcoded dimensions come last and come per feature. Start with the screens that are known to look wrong on tablet.

7.6 Impact on future maintenance

MediaQuery.of(context) subscribes to the whole MediaQuery object, not just the field being read. 240 of the 242 MediaQuery reads in this codebase go through .of(context) rather than the field-scoped MediaQuery.sizeOf(context) that exists specifically to avoid this. A widget that reads .size.width once, purely to decide phone or tablet, rebuilds again every time the keyboard opens, because opening the keyboard changes the view insets, which are also part of what .of(context) is subscribed to. On a form screen this means a layout check that has nothing to do with the keyboard reruns on every field focus.

The four disagreeing breakpoint definitions compound this rather than merely adding to it. Two adjacent widgets built against different helpers can each decide a given width differently, so a screen that looks correct on the device it was built on can show the wrong layout at a width nobody tested, and there is no single place to check that would catch it before a user does.

8. Models everywhere, generated, and never raw maps

8.1 Where we are

Every JSON boundary in this app is parsed by hand. There are **164 hand-written fromJson factories and 105 hand-written toJson methods across 108 files**. Not one of them is generated.

`build_runner ^2.1.4` is already a dev dependency. It is there for `isar_generator` only. All six `.g.dart` files in the repo are Isar collection code for the communication feature, totalling 33,282 lines. There is no `json_serializable`, no `json_annotation` and no `freezed` anywhere in pubspec. The code generation pipeline exists and is running. It has simply never been pointed at the models.

Measured	Count	Comment
`Hand-written fromJson factories`	**164**	Across 108 files
`Hand-written toJson methods`	**105**	
`Generated JSON models`	**0**	All 6 .g.dart files are Isar, not JSON
`Unchecked casts of the form json[key] as Type`	**132**	Throws on a null or a type change from the backend
`Occurrences of as String? ?? ''`	321	The defaulting rule, restated by hand 321 times
`Occurrences of as int? ?? 0`	171	Same
`Occurrences of ?.toString() ??`	113	A third way of doing the same thing
`(json[key] as num?)?.toDouble()`	51	A fourth
`Private parse closures inside factory bodies`	12	Cannot be tested or reused
`fromJson taking dynamic instead of Map<String, dynamic>`	2	No type contract at the boundary at all

8.2 What hand-parsing has already cost

a. The same field parsed two different ways in one file

`lib/data/features/attendance/models/student_attendance_models.dart` is 1,274 lines and holds 32 fromJson factories. Three of them read `signedInBy`. Two handle the backend alias. One does not.

```
// line 90,  StudentAttendanceDailyItemModel
signedInBy: (json['signedInBy'] ?? json['signedInByName']) as String?,

// line 661, StudentAttendanceTransferItemModel
signedInBy: (json['signedInBy'] ?? json['signedInByName']) as String?,

// line 434, StudentAttendanceCustomItemModel   <-- alias silently dropped
signedInBy: json['signedInBy'] as String?,
```

When the backend sends `signedInByName`, the daily view shows who signed the student in and the custom-range view shows nothing. Same screen family, same file, different answer. Nobody wrote that bug on purpose. It is what happens when the same contract is retyped 32 times.

b. The same wire field parsed into two different types

In the same file, `signedInTime` becomes a `StudentAttendanceTimeModel` in one model and a `String?` in another, via a private closure defined inside the factory body.

```
// line 87
signedInTime: json['signedInTime'] != null
    ? StudentAttendanceTimeModel.fromJson(json['signedInTime'])
    : null,

// line 412, a closure that exists only inside this one factory
String? _parseTime(dynamic value) {
  if (value == null) return null;
  if (value is String) return value;
  if (value is Map<String, dynamic>) {
    final hour   = value['hour']   as int? ?? 0;
    final minute = value['minute'] as int? ?? 0;
    final second = value['second'] as int? ?? 0;
    return '${h}:${m}:${s}';
  }
  return value.toString();
}
signedInTime: _parseTime(json['signedInTime']),
```

Twelve of these private closures exist across the codebase. None can be unit tested, none can be reused, and each is a separate opinion about what the backend sends.

c. Parsing that leaked out of the models folder

Parsing is not even confined to model files. Six datasources parse JSON inline:

Datasource	fromJson calls
`data/features/attendance/datasources/attendance_remote_datasource.dart (1,915 lines)`	**21**
`data/features/student/datasources/student_remote_datasource.dart (1,985 lines)`	**19**
`data/features/staff/datasources/staff_remote_datasource.dart (1,223 lines)`	18
`data/features/billing/datasources/invoice_remote_datasource.dart`	15
`data/features/billing/datasources/auto_pay_remote_datasource.dart`	10
`data/features/scheduling/datasources/staff_shift_remote_datasource.dart`	7

d. Parsing that reached the domain layer

Rule 8 of the May 2026 standard was "no fromJson/toJson on domain entities". Two violations survive:

```
// domain/features/classroom/entities/ratio_monitoring_classroom.dart:69
```

```
factory ClassroomAttendanceSummaryEntry.fromJson(Map<String, dynamic> json) => ...

// domain/features/auth/usecases/fetch_kiosk_user_details_usecase.dart:40
final model = UserDetailsResponseModel.fromJson(response);
   // a use case parsing a data-layer model directly
```

8.3 Maps used as data carriers

Separate from parsing, and worse. A raw `Map<String, dynamic>` is used as the data type itself, passed between layers instead of a model. There are **1,802 references to Map<String, dynamic> outside generated files**, and they are not confined to the serialisation boundary.

Layer	References	Expected
`lib/data`	1,216	Legitimate at the parse boundary only. Section 8.1 covers the rest.
`lib/presentation`	**413**	Should be zero. The UI has entities, not maps.
`lib/domain`	**73**	Should be zero. This is the pure layer.
`lib/shared and lib/core`	75	Mostly storage helpers, some legitimate.

Broken out by how the map is used: 427 as function parameters, 84 as return types, 82 as declared fields. Those are not parsing. Those are a map standing in for a type that should exist.

a. Domain contracts that traffic in maps

The domain layer is meant to be pure typed Dart. These contracts take and return untyped maps.

```
// domain/features/activity/repositories/activity_repository.dart:76, 77
List<Map<String, dynamic>> parameterValues = const [],
List<Map<String, dynamic>> attachments     = const [],

// domain/features/activity/repositories/activity_repository.dart:123
Future<List<Map<String, dynamic>>> createSharedActivities({ ... });
   // a domain contract RETURNING untyped maps

// domain/features/attendance/repositories/attendance_repository.dart:348-378
Future<AttendanceActionResult> submitStudentSignIn({
  required Map<String, dynamic> payload,
  required bool isParent,
});
   // and the same for signOut, transfer, staffSignIn, staffSignOut,
   // staffTransfer. Six methods, six opaque payloads.
```

b. The model already exists

This is the part worth being blunt about. In both cases above, somebody already wrote the model, and the map is used anyway.

Model that exists	What the contract takes instead

`ActivityTypeParameterModel`
`data/features/activity/models/activity_type_details_model.dart:96`
plus the entity ActivityTypeParameter at
`domain/features/activity/entities/activity_type_details.dart:67`
	`List<Map<String, dynamic>> parameterValues`
	`activity_repository.dart:76, 116, 134`

`MarkAbsentRequestModel` `data/features/attendance/models/student_absence_record_models.dart:3` *and StaffMarkAbsentRequestModel alongside it*	`required Map<String, dynamic> payload` `attendance_repository.dart:348, 354, 360, 365, 372, 378`

The mark-absent flow has a request model. The sign-in, sign-out and transfer flows, in the same feature and the same repository, take a bare map. The pattern was established and then not followed, which is the whole of this mandate in one file.

c. Where those maps are built

In a view. `presentation/features/attendance/forms/student_attendance_forms/student_sign_in_form.dart` constructs the payload by hand, mutates it, then reads values back out of it.

```
// line 601
final payload = <String, dynamic>{ ... };

// line 656
payload.addAll(<String, dynamic>{
  'attendanceDate': attendanceDate,
  'staffCode': staffCode,
  'classroomCode': defaultClassroomCode,
  'classroomCodeByStudentCode': classroomCodeByStudentCode,
  'occurredAt': occurredAt.toUtc().toIso8601String(),
});

// line 668
payload['method'] = 'PIN';

// line 683, reading back out of the map it just built
final staffCode = (payload['staffCode'] ?? '').toString().trim();

// line 695, handed to the domain repository as-is
payload: payload,
```

That map crosses presentation, domain and data with no type at any layer. Misspell `'classroomCode'` and it compiles, ships, and fails at the server. Rename the field backend-side and nothing in the app fails to build. The compiler has been asked to check nothing at all on the highest-traffic write path in the product.

8.4 The rule

- Every API payload is represented by a model class in `data/features/<x>/models/`. No inline map access anywhere else.

- Models are annotated with `@JsonSerializable()` and their `fromJson` / `toJson` bodies are generated. No hand-written serialisation.
- Field-level quirks (aliases, date formats, numeric coercion) are declared as annotations, not written as inline expressions. `@JsonKey(name: 'signedInByName')` and `@JsonKey(readValue: ...)` handle every case currently done by hand.
- Defaulting is declared once per field with `@JsonKey(defaultValue: ...)`. The 321 + 171 + 113 + 51 restatements of the four defaulting idioms collapse into annotations.
- No `Map<String, dynamic>` in `lib/domain` or `lib/presentation`. Not as a parameter, not as a return type, not as a field. Maps stop at the datasource.
- If an entity has a model, that model is the type used everywhere it travels. Passing a map for a shape that already has a model class is the specific thing this mandate exists to stop.
- Request bodies get request models too, not just responses. `MarkAbsentRequestModel` is the pattern; sign-in, sign-out and transfer need the same.
- A view never builds a wire payload. It hands typed values to a cubit, the cubit hands a model to a use case, and `toJson()` happens once, in the datasource.
- Datasources call `Model.fromJson(response)` and return the model. They do not read keys off a map.
- Domain entities have no serialisation. Mappers convert model to entity, which is what the `mappers/` folder is for. Only 2 of 19 features have mappers today.
- A model change is a generator run, not an edit. `.g.dart` files are committed so CI and fresh clones do not need a build step before analysing.

8.5 Worked example

The daily attendance model, converted. Every quirk currently written by hand has an annotation equivalent.

WRONG, as it stands today

```
class StudentAttendanceDailyItemModel {
  const StudentAttendanceDailyItemModel({ ... });
  final String? studentCode;
  final String  studentName;
  final bool    isExpectedToday;
  final List<String> tagCodes;

  factory StudentAttendanceDailyItemModel.fromJson(Map<String, dynamic> json) {
    return StudentAttendanceDailyItemModel(
      studentCode:     json['studentCode'] as String?,
      studentName:     json['studentName'] as String? ?? '',
      isExpectedToday: json['isExpectedToday'] as bool? ?? false,
      signedInBy:      (json['signedInBy'] ?? json['signedInByName']) as String?,
      tagCodes: (json['tagCodes'] as List<dynamic>?)
          ?.map((e) => e.toString()).toList() ?? const [],
      // ...30 more fields, restated in 31 other factories
    );
  }
}
```

RIGHT

```
import 'package:json_annotation/json_annotation.dart';
```

```
part 'student_attendance_daily_item_model.g.dart';

@JsonSerializable()
class StudentAttendanceDailyItemModel {
  const StudentAttendanceDailyItemModel({
    this.studentCode,
    this.studentName = '',
    this.isExpectedToday = false,
    this.signedInBy,
    this.tagCodes = const [],
  });

  final String? studentCode;

  @JsonKey(defaultValue: '')
  final String studentName;

  @JsonKey(defaultValue: false)
  final bool isExpectedToday;

  // the alias, declared once instead of remembered three times
  @JsonKey(readValue: _signedInBy)
  final String? signedInBy;

  @JsonKey(defaultValue: <String>[])
  final List<String> tagCodes;

  factory StudentAttendanceDailyItemModel.fromJson(Map<String, dynamic> json) =>
      _$StudentAttendanceDailyItemModelFromJson(json);

  Map<String, dynamic> toJson() =>
      _$StudentAttendanceDailyItemModelToJson(this);

  static Object? _signedInBy(Map map, String key) =>
      map['signedInBy'] ?? map['signedInByName'];
}
```

What the generator buys that a careful developer does not. It cannot forget a field. Add signedInByName handling once and every model that declares the field gets it. Rename a field and the build fails rather than silently returning null at runtime. The 132 unchecked casts become generated null-safe reads. And a reviewer reads a field list instead of auditing 40 lines of map access for the one line that differs from the other 31 factories.

8.6 Migration

108 files is too many for one sweep, but the setup cost is close to zero because build_runner is already wired.

- Add `json_annotation` to dependencies and `json_serializable` to dev_dependencies. build_runner is already there for Isar, so there is no new pipeline and no new CI step.
- Convert one feature end to end first, as the reference. `billing` or `newsletter` are the right size. Do not start with attendance.
- Then attendance, student and staff, in that order. They hold the most parsing (32, 19 and 22 factories) and the most duplicated field handling, so they return the most.

- While converting a feature, pull its inline parsing out of the datasource and into the model. That closes 8.2c in the same pass.
- Type the six attendance payloads first, independently of the generator work. `MarkAbsentRequestModel` shows the shape, the change is contained to one repository and its callers, and it removes untyped data from the busiest write path in the product.
- Add the missing `mappers/` folder per feature as you go. Generated models make mappers trivial, and 17 of 19 features still need one.
- Delete the two domain-layer parsers named in 8.2d once their feature has mappers.

One decision to make up front. json_serializable and freezed both solve this. freezed additionally gives immutability, copyWith and sealed unions, which would also serve the sealed cubit states in Section 2, but it generates more code and is a bigger change to how models are declared. json_serializable is the smaller step and converts a model without changing its shape. Pick one before the first feature is converted, not after.

8.7 Impact on future maintenance

json['signedInBy'] gives an editor nothing to check: no autocomplete after the bracket key, no declared type, and no warning if the backend renames or removes the field. model.signedInBy autocompletes, carries a type, and turns a renamed field into a build failure instead of a screen that quietly stops showing a value. That gap is exactly how the alias-handling logic in student_attendance_models.dart could drift across three copies of the same read without anyone noticing, until two attendance views disagreed about who signed a student in.

The same gap applies going forward, not just to what already exists. Every new endpoint added to this codebase is another hand-written parser choosing its own defaulting convention from among the four already in use, and a mismatch between two of them will not surface until the screen that depends on it is opened and looked at.

9. Making it stick

The May 2026 standard listed 43 findings. Eleven are fixed today. The eleven that got fixed were the ones one person could finish in an afternoon. The ones that needed every developer to change habit did not move, and several got measurably worse.

The difference was not effort or intent. It was that **nothing failed when a rule was broken**.

9.1 The blocker

`analysis_options.yaml` exists in the repo root. All 45 lines of it are commented out, including `include: package:flutter_lints/flutter.yaml`. flutter analyze currently enforces nothing beyond the Dart compiler defaults.

The cost of turning it on is smaller than it looks, because it was measured:

Configuration	Issues	Comment
`Today (everything commented out)`	136	Compiler defaults only. 116 in lib/, 20 in test/.
`flutter_lints, generated files excluded`	**101**	One afternoon of work. This should go in immediately.
`Plus the 5 rules the May doc proposed`	**1,408**	942 discarded_futures, 315 prefer_single_quotes, 49 flutter_style_todos, 102 other.

The 942 `discarded_futures` are not style. Each one is an async call whose failure is silently dropped. Together with the 585 `catch (_)` blocks, that is 1,527 places where something can fail and nobody finds out.

9.2 The other blocker

26 of 206 tests fail on develop at the reference commit. The branch every developer works from is red, and has been long enough that nobody notices.

Failing test	What it is telling you
`staff_code_resolver_test.dart` *"regression for the schoolCode vs candidateSchoolCode bug"*	A test written specifically to stop a bug coming back is failing. The bug is back.
`api_paths_uat_test.dart (2 tests)`	Produces /api/v1/staff/staff/my-profile/... with a doubled path segment, against a devtunnel host rather than the UAT gateway.
`students_and_family_report_params_test.dart`	Expects ['Inactive'], gets ['INACTIVE']. A live API filter with the wrong casing.
`feature_flag_cubit_test.dart`	Does not compile. See Section 3.4.
`private_mode_flow_test.dart (5 tests)`	Integration flow, failing.
16 further failures	Across reports and widget tests.

Test noise is part of why this went unnoticed. `AppLogger` is gated on `kDebugMode`, which is true under flutter test, so the suite emits hundreds of coloured DEBUG lines and the failures scroll past. Silence the logger in the test harness.

9.3 Minimum gates

Every one of the eight mandates needs a gate, or it is a suggestion. These are the gates, in the order they should land.

#	Gate	Enforces
1	**Uncomment analysis_options.yaml with flutter_lints. CI fails on any issue.**	The floor. Nothing else below works without it.
2	CI runs flutter test. Red build blocks merge.	Requires fixing the 26 failures first. Silence AppLogger under test.
3	Custom lint or CI grep: no MaterialPageRoute or Navigator.push under lib/presentation.	Mandate 1
4	CI grep on the diff: added setState under lib/presentation needs explicit reviewer sign-off.	Mandate 2
5	CI grep: no relative import crossing a folder boundary; tests import package:wowcare/...	Mandate 3
6	CI grep: no Color(0x, no Colors.<name>, no inline TextStyle(, no fontFamily: outside core/theme/.	Mandate 4
7	CI grep: no serviceLocator<, no *DataSource, no ApiClient, no Dio under lib/presentation except core/router/.	Mandate 5
8	CI check: no duplicate public class names across lib/.	Mandate 6
9	CI grep: no screen-width literal comparison outside core/responsive/.	Mandate 7
10	CI check: no hand-written fromJson or toJson outside a .g.dart file, and no Map<String, dynamic> in lib/domain or lib/presentation.	Mandate 8
11	File size ceiling: new files over 400 lines fail. Existing files over 400 must not grow.	Stops the four God Objects that regrew after May.

Gates 3 through 10 are grep rules on the diff. They take an afternoon to write and they are the difference between this document and the last one.

9.4 What is deliberately not in this document

These are real problems, they were measured, and they do not fit the eight mandates. They need owners and dates of their own.

- **Test coverage.** 31 test files for 1,389 source files. 1 use case test for 224 use cases. No mocktail, mockito or bloc_test in pubspec.
- **Environment configuration.** ENVIRONMENT defaults to LOCAL, and LOCAL resolves to two named developers' personal devtunnels and a personal ngrok account hardcoded in lib/core/util/api_constants.dart. Backends are switched by editing _activeDev and committing it. There is no .env, no dotenv, no .env.example.

- **Accessibility.** Zero Semantics, semanticLabel or MergeSemantics in 1,389 files. Flagged twice in May, unchanged.
- **Localisation.** No l10n.yaml, no .arb files, no flutter_localizations. Mandate 4 makes this a later mechanical job rather than a rewrite, which is the main reason to do Mandate 4 properly.
- **God Objects.** activity_daily_tab.dart is 7,065 lines and grew since May. 69 non-generated files exceed 1,000 lines. Gate 10 stops the growth; splitting them is separate work.
- **Dependency hygiene.** rename is still a runtime dependency, moment_dart and intl both ship, flutter_tindercard comes from an unpinned git URL. All three were named in May.

Closing note. The foundation this document asks for is mostly already built. ExpandableInfoCard is written. The TextTheme is defined and wired. AppColors has 242 lines of tokens. The domain layer has 224 use cases. SizerInit solves the LayoutBuilder placement problem correctly. GoRouter has 56 routes configured. build_runner is already wired and running.

What is missing is not architecture. Using the right thing is optional today, and slightly slower than not bothering. Fix that first, in Section 9, and the other eight sections become achievable.